



Coffee Shop Analysis

Index



Objective (Coffee Shop DB)

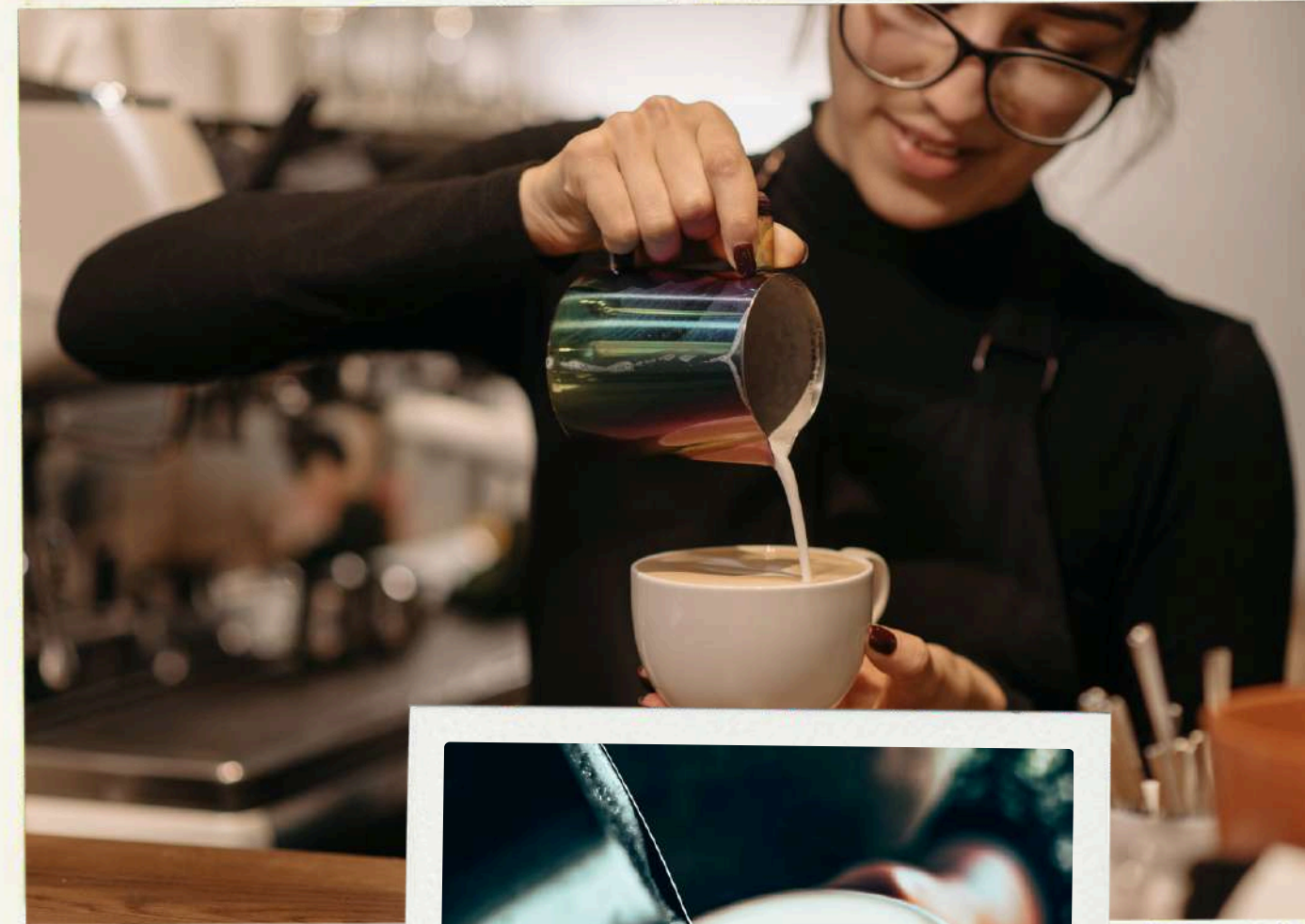
- 1. Database & Tables**
- 2. Structure Of Table (ER-Diag)**
- 3. Constraints (PK, FK, etc.)**
- 4. CRUD Operations**
- 5. Filtering Data (WHERE, LIKE, Operators)**
- 6. Grouping & Aggregation**
 - **GROUP BY**
 - **HAVING**
- 7. Sorting & Pagination**
 - **ORDER BY**
 - **LIMIT**
 - **OFFSET**
- 8. Joins (INNER, LEFT, RIGHT)**
- 9. Advanced SQL**
 - **CTE**
 - **Index**
- 10. Conclusion (Coffee Shop Analysis)**



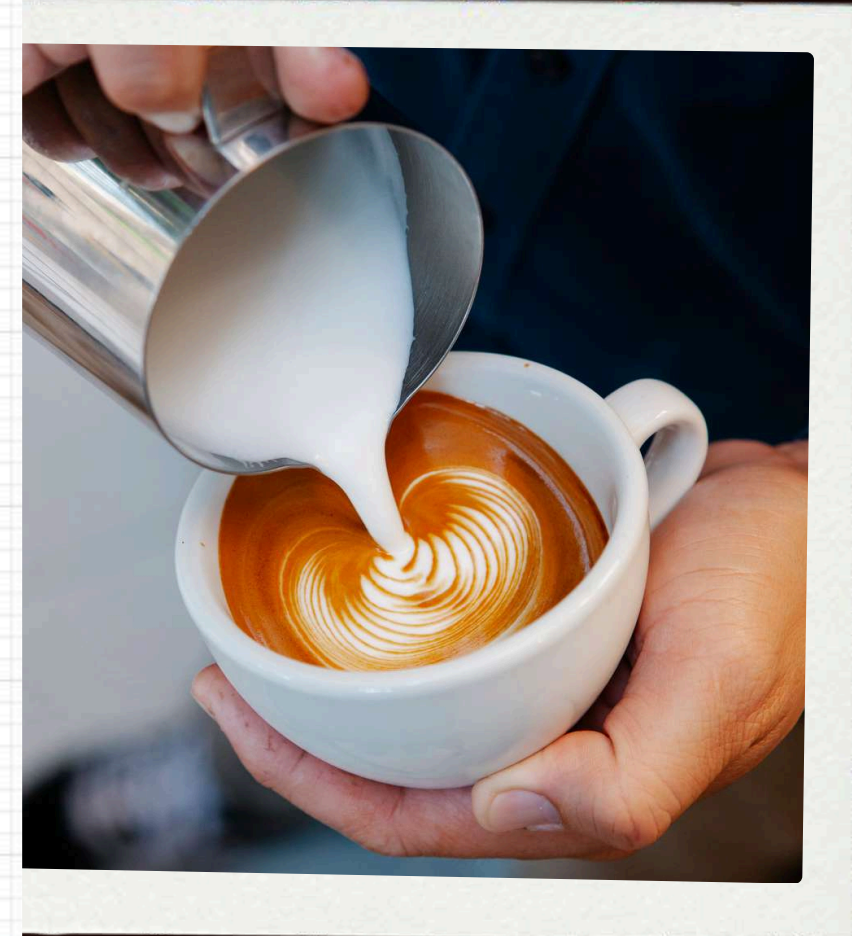
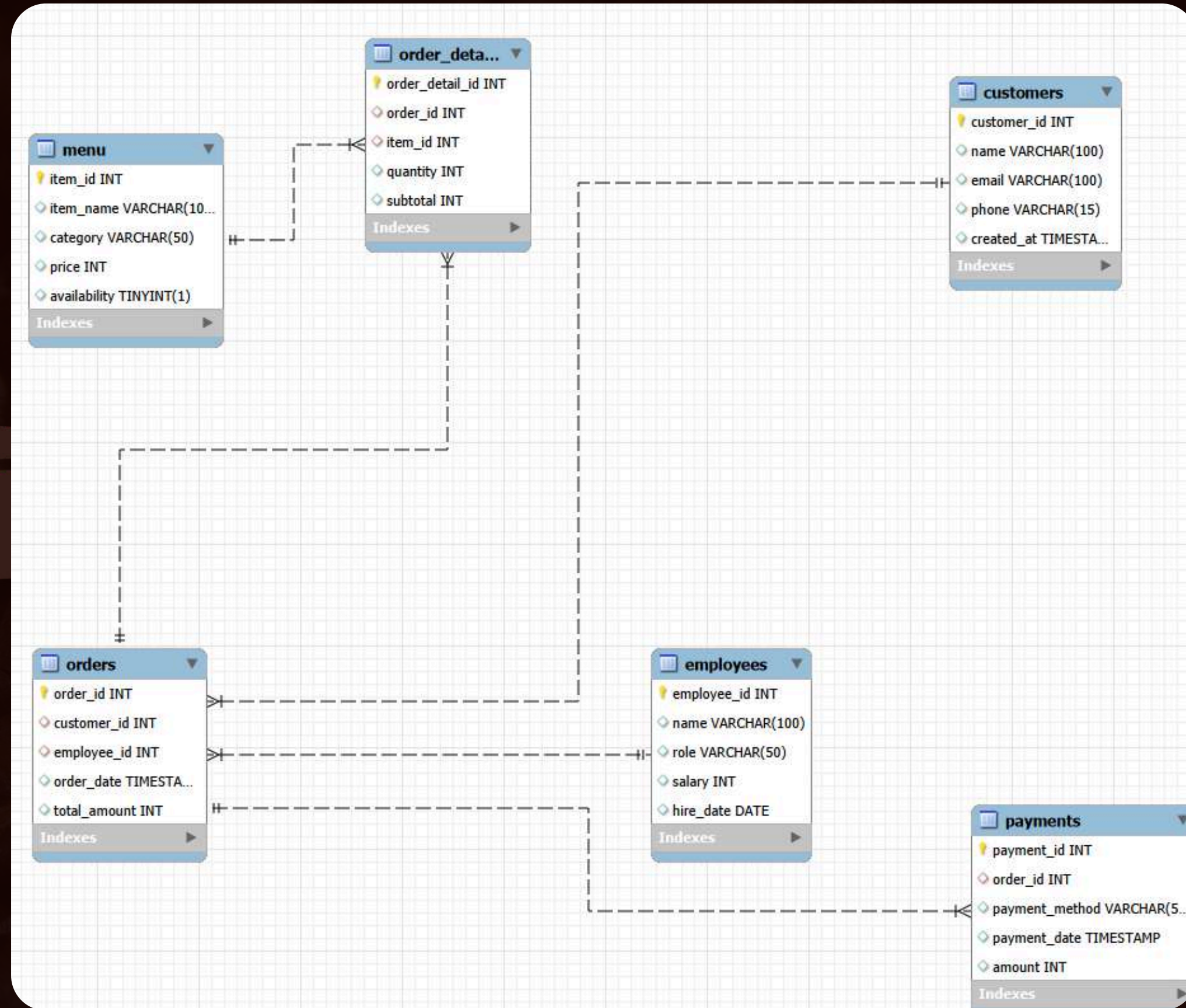
Objective:



- This project aims to analyze coffee shop sales data using SQL.
- By applying various SQL queries, the analysis extracts key insights such as total orders, revenue generated, most popular coffee types, and customer preferences.
- The objective is to understand business performance, identify trends, and support data-driven decision-making for improving sales and customer satisfaction.



ER- DIAGRAM



1. Create & Use Database



A database is a structured collection of data. Tables store data in rows (records) and columns (fields).

```
CREATE DATABASE CoffeeShopDB;  
USE CoffeeShopDB;
```

```
CREATE TABLE Customers (  
  customer_id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100),  
  email VARCHAR(100) UNIQUE,  
  phone VARCHAR(15),  
  created_at    TIMESTAMP    DEFAULT  
CURRENT_TIMESTAMP  
);
```


1. Create & Use Database



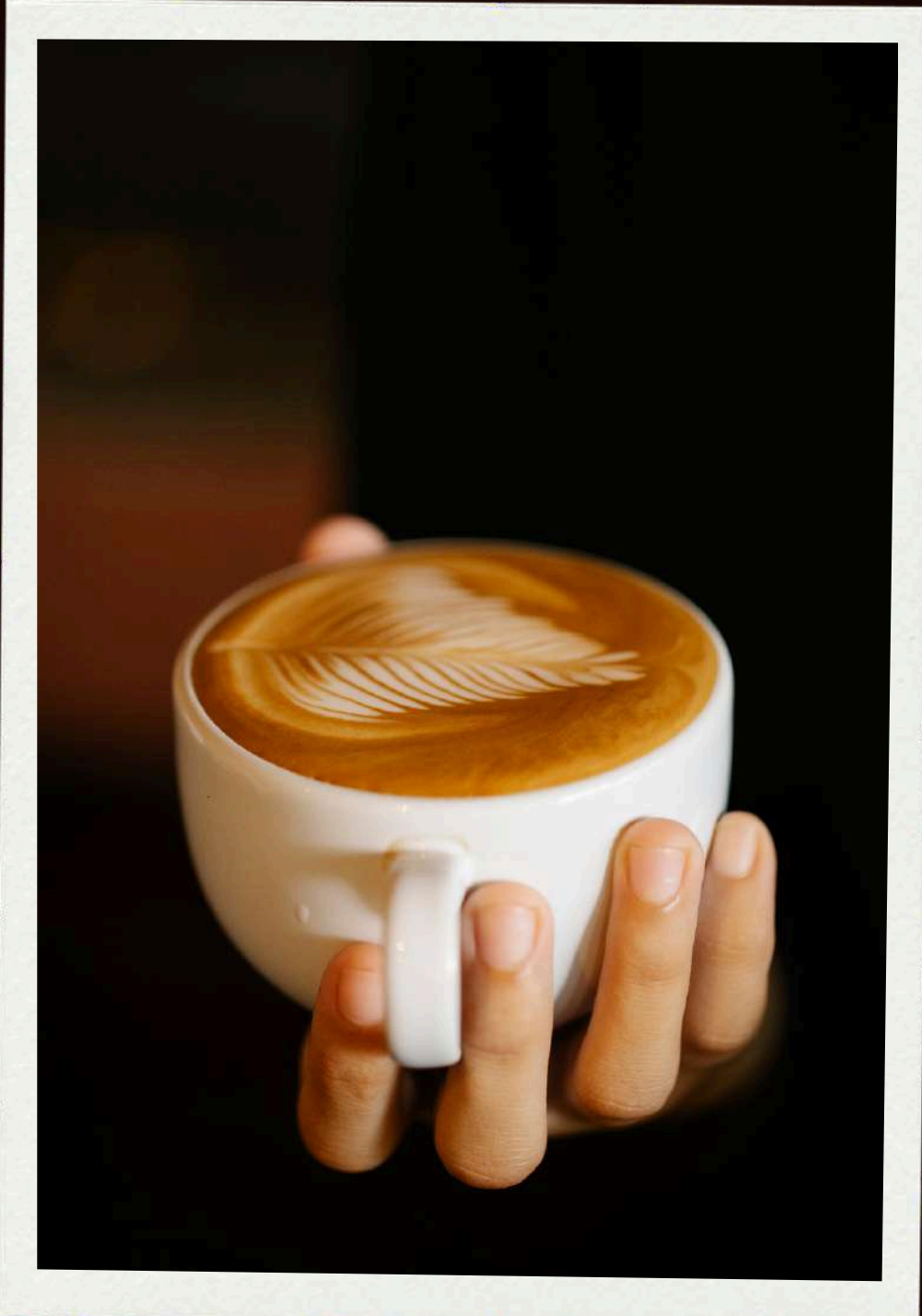
```
4 • CREATE TABLE Customers (  
5     customer_id INT PRIMARY KEY AUTO_INCREMENT,  
6     name VARCHAR(100),  
7     email VARCHAR(100) UNIQUE,  
8     phone VARCHAR(15),  
9     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
10 );
```

Result Grid | Filter Rows: | Edit: | Export/Import:

	customer_id	name	email	phone	created_at
•	1	Amit Verma	amit.verma@example.com	9871234567	2026-04-11 15:11:49
	2	Neha Gupta	neha.gupta@example.com	9765432109	2026-04-11 15:11:49
	3	Rohit Mehta	rohit.mehta@example.com	9654321098	2026-04-11 15:11:49
	4	John Doe	john.doe@example.com	9876543210	2026-04-11 15:11:49
	5	Jane Smith	jane.smith@example.com	9123456780	2026-04-11 15:11:49
	6	Rahul Sharma	rahul.sharma@example.com	9988776655	2026-04-11 15:11:49
	7	Priya Patel	priya.patel@example.com	9090909090	2026-04-11 15:11:49
	8	Priya Jhoshi	priya.jhosh2@example.com	9090905336	2026-04-11 15:11:49

Example :
Customers, Orders, and Products tables store coffee shop sales data.

2. Structure Of Table



```
15 • desc Customers;  
16
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	Field	Type	Null	Key	Default	Extra
▶	customer_id	int	NO	PRI	NULL	auto_increment
	name	varchar(100)	YES		NULL	
	email	varchar(100)	YES	UNI	NULL	
	phone	varchar(15)	YES		NULL	
	created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

Customers Table :

This Table Stores customer info with unique email and auto timestamp when created.

2. Structure Of Table



```
42 • desc Employees;
```

```
43
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	Field	Type	Null	Key	Default	Extra
▶	employee_id	int	NO	PRI	NULL	auto_increment
	name	varchar(100)	YES		NULL	
	role	varchar(50)	YES		NULL	
	salary	int	YES		NULL	
	hire_date	date	YES		NULL	

Employees Table :

This Table Stores staff details like ID, name, role, salary, and hire date.

2. Structure Of Table



```
61 • select * from Menu;
```

```
62 • desc Menu;
```

```
63
```

Result Grid

Filter Rows:

Export:

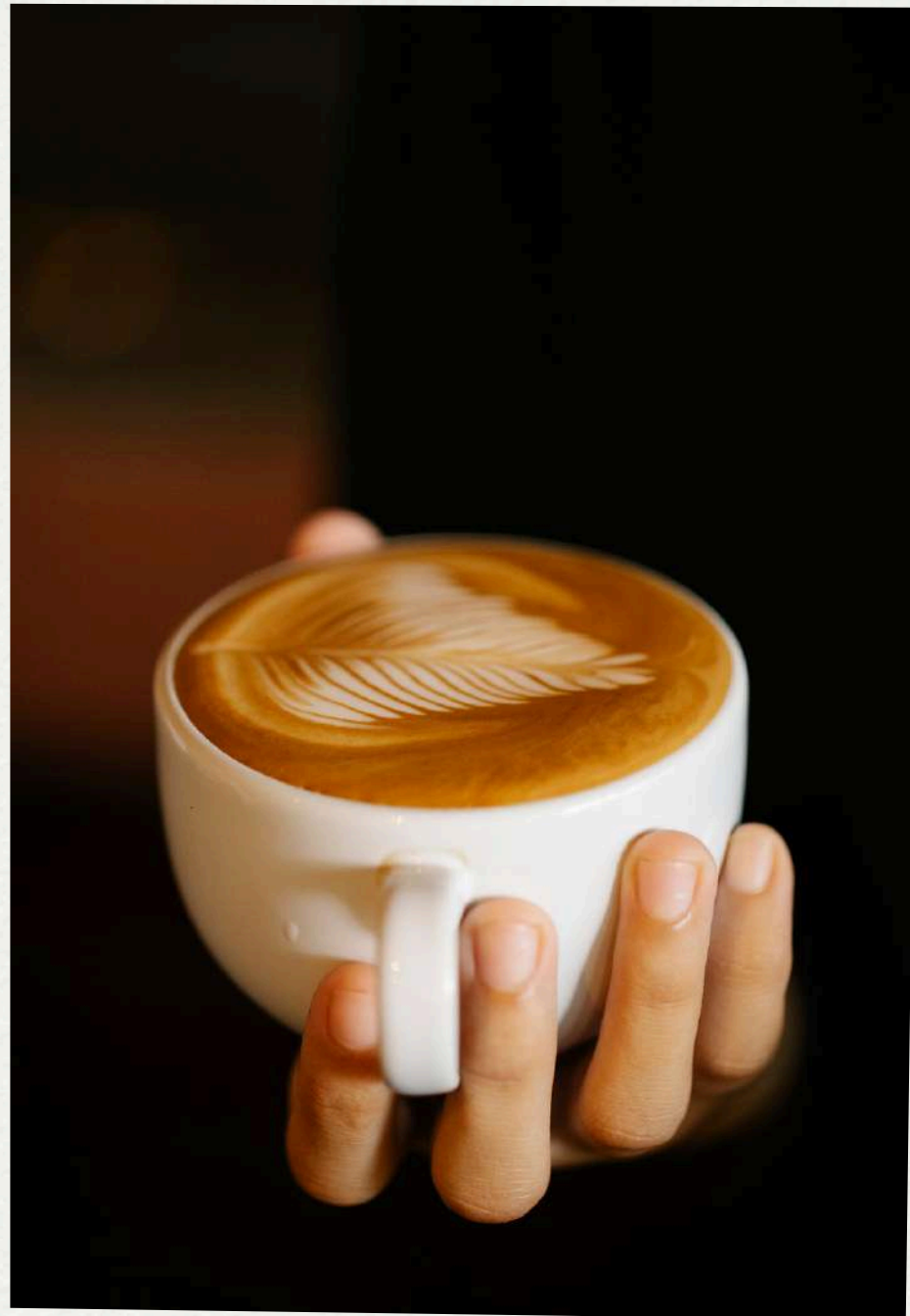
Wrap Cell Content:

	Field	Type	Null	Key	Default	Extra
▶	item_id	int	NO	PRI	NULL	auto_increment
	item_name	varchar(100)	YES		NULL	
	category	varchar(50)	YES		NULL	
	price	int	YES		NULL	
	availability	tinyint(1)	YES		1	

Menu Table :

This Table Stores menu items with ID, name, category, price, and availability (1 = available).

2. Structure Of Table



```
84 • select * from Orders;
```

```
85 • desc Orders;
```

```
86
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	Field	Type	Null	Key	Default	Extra
▶	order_id	int	NO	PRI	NULL	auto_increment
	customer_id	int	YES	MUL	NULL	
	employee_id	int	YES	MUL	NULL	
	order_date	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED
	total_amount	int	YES		NULL	

Orders Table :

This Table Stores each order with customer, employee, date, and total amount. Links to Customers and Employees.

2. Structure Of Table



```
95 • desc Order_Details;
```

```
96
```

```
97 • INSERT INTO Order_Details (order_id, item_id,
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	Field	Type	Null	Key	Default	Extra
▶	order_detail_id	int	NO	PRI	HULL	auto_increment
	order_id	int	YES	MUL	HULL	
	item_id	int	YES	MUL	HULL	
	quantity	int	YES		HULL	
	subtotal	int	YES		HULL	

Order Details Table :

This Table Stores items inside each order (which menu item, quantity, subtotal). Links to Orders and Menu.

2. Structure Of Table



```
129 • desc Payments;
```

Field	Type	Null	Key	Default	Extra
payment_id	int	NO	PRI	NULL	auto_increment
order_id	int	YES	MUL	NULL	
payment_method	varchar(50)	YES		NULL	
payment_date	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED
amount	int	YES		NULL	

Payments Table :

This Table Stores payment info for each order (method, date, amount). Links to Orders.



3. Constraints

- 01 **PRIMARY KEY (order_id)** : Uniquely identifies each record in the table (no duplicates, no NULL).
- 02 **NOT NULL (customer_id)** : Ensures the column cannot have NULL (empty) values.
- 03 **CHECK (total_amount > 0)** : Ensures values meet a specific condition.
- 04 **DEFAULT (order_date = CURRENT_DATE)** : Assigns a default value if none is provided.
- 05 **FOREIGN KEY (customer_id)** : Links one table to another and maintains referential integrity.





3. *Constraints*

```
CREATE TABLE Orders (  
  order_id INT,  
  customer_id INT NOT NULL,  
  total_amount DECIMAL(8,2),  
  order_date DATE DEFAULT CURRENT_DATE,  
  CONSTRAINT pk_order PRIMARY KEY  
  (order_id),  
  CONSTRAINT chk_amount CHECK  
  (total_amount > 0),  
  CONSTRAINT fk_customer FOREIGN KEY  
  (customer_id)  
  REFERENCES Customers(customer_id) );
```





4. CRUD Operations



Create

Add new data

```
INSERT INTO Customers (customer_id,  
name, city)  
VALUES (1, 'Rahul', 'Mumbai');
```



Read

Retrieve data

```
SELECT * FROM Customers;
```



Update

Modify existing data

```
UPDATE Customers  
SET city = 'Pune'  
WHERE customer_id = 1;
```



Delete

Remove data

```
DELETE FROM Customers  
WHERE customer_id = 1;
```


4. CRUD Operations



```
150 • UPDATE Customers
151     SET name = 'Rahul Joshi'
152     WHERE Customer_id = 3;
153
154 • select * from Customers;
```

Result Grid

Filter Rows:

Edit:

Export/Import:

	customer_id	name	email	phone	created_at
	1	Amit Verma	amit.verma@example.com	9871234567	2026-04-11 15:11:49
	2	Neha Gupta	neha.gupta@example.com	9765432109	2026-04-11 15:11:49
	3	Rahul Joshi	rohit.mehta@example.com	9654321098	2026-04-11 15:11:49
	4	John Doe	john.doe@example.com	9876543210	2026-04-11 15:11:49
	5	Jane Smith	jane.smith@example.com	9123456780	2026-04-11 15:11:49
	6	Rahul Sharma	rahul.sharma@example.com	9988776655	2026-04-11 15:11:49

```
155 • DELETE FROM Customers
156     WHERE Customer_id = 8;
157 • select * from Customers;
```

Result Grid

Filter Rows:

Edit:

Export/Import:

	customer_id	name	email	phone	created_at
▶	1	Amit Verma	amit.verma@example.com	9871234567	2026-04-11 15:11:49
	2	Neha Gupta	neha.gupta@example.com	9765432109	2026-04-11 15:11:49
	3	Rahul Joshi	rohit.mehta@example.com	9654321098	2026-04-11 15:11:49
	4	John Doe	john.doe@example.com	9876543210	2026-04-11 15:11:49
	5	Jane Smith	jane.smith@example.com	9123456780	2026-04-11 15:11:49
	6	Rahul Sharma	rahul.sharma@example.com	9988776655	2026-04-11 15:11:49
	7	Priya Patel	priya.patel@example.com	9090909090	2026-04-11 15:11:49
	NULL	NULL	NULL	NULL	NULL





5. Filtering Data (WHERE, LIKE, Operators)



WHERE

Filter rows based on a condition

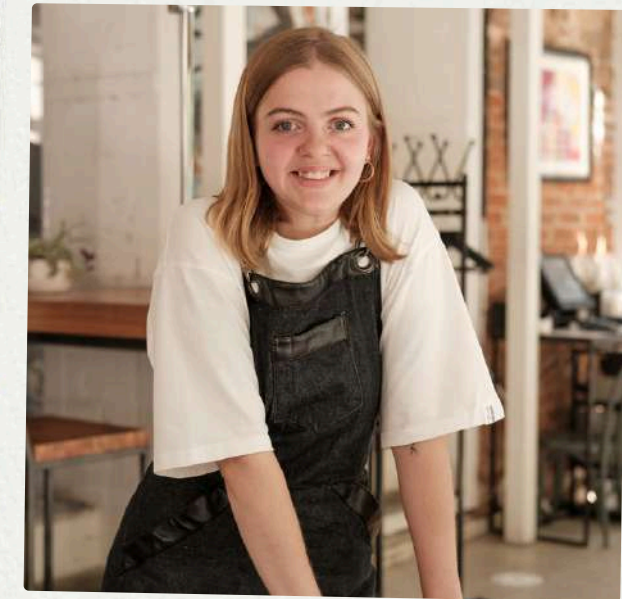
```
SELECT * FROM Orders  
WHERE total_amount > 500;
```



LIKE

Filter rows by pattern matching (e.g., names starting with 'R')

```
SELECT * FROM Customers  
WHERE name LIKE 'R%';
```



OPERATORS

Use =, >, <, BETWEEN, IN to refine queries

```
SELECT * FROM Orders  
WHERE total_amount  
BETWEEN 200 AND 1000;
```




5. Filtering Data (WHERE, LIKE, Operators)

```
160 • SELECT *
161 FROM Employees
162 WHERE salary > 50000
163 AND name LIKE '%a%';
```

Result Grid | Filter Rows: Edit:

	employee_id	name	role	salary	hire_date
▶	1	Amit Sharma	Manager	75000	2022-01-15
	2	Neha Verma	Developer	60000	2023-03-10
	4	Priya Singh	HR	55000	2020-11-05
	5	Karan Patel	Developer	62000	2022-06-18
	8	Anjali Desai	Analyst	53000	2022-08-25
✱	NULL	NULL	NULL	NULL	NULL



6. Grouping & Aggregation



- **GROUP BY**

Group rows that have the same values in one or more columns, allowing aggregate functions like SUM, COUNT, AVG, MAX, MIN to calculate summary statistics for each group.

- `SELECT customer_id, SUM(total_amount) AS total_spent FROM Orders GROUP BY customer_id;`

- **HAVING**

Filter groups after aggregation (e.g., customers who spent >1000)

- Filter groups after aggregation (e.g., customers who spent >1000)

6. Grouping & Aggregation



```
166 • SELECT customer_id, SUM(total_amount) AS total_spent
167     FROM Orders
168     GROUP BY customer_id
169     HAVING SUM(total_amount) > 700;
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:

	customer_id	total_spent
▶	1	1230



7. Sorting & Pagination



ORDER BY

Sort results by a column
(ascending/descending)

```
SELECT * FROM Orders  
ORDER BY total_amount DESC;
```



LIMIT

Show only a fixed number of
rows

```
SELECT * FROM Orders  
LIMIT 5;
```



OFFSET

Skip a number of rows (used
with LIMIT for pagination)

```
SELECT * FROM Orders  
LIMIT 5 OFFSET 5;
```




7. Sorting & Pagination

```
175 • SELECT *
176 FROM Employees
177 LIMIT 3;
```

Result Grid

Filter Rows:

Edit:

	employee_id	name	role	salary	hire_date
	1	Amit Sharma	Manager	75000	2022-01-15
	2	Neha Verma	Developer	60000	2023-03-10
	3	Rahul Mehta	Designer	50000	2021-07-22
	NULL	NULL	NULL	NULL	NULL

```
171 • SELECT *
172 FROM Employees
173 ORDER BY salary DESC;
```

Result Grid

Filter Rows:

Edit:

	employee_id	name	role	salary	hire_date
▶	1	Amit Sharma	Manager	75000	2022-01-15
	5	Karan Patel	Developer	62000	2022-06-18
	2	Neha Verma	Developer	60000	2023-03-10
	4	Priya Singh	HR	55000	2020-11-05
	8	Anjali Desai	Analyst	53000	2022-08-25
	3	Rahul Mehta	Designer	50000	2021-07-22
	6	Sneha Gupta	Tester	48000	2023-09-01
	7	Rohit Jain	Support	40000	2021-12-12
	NULL	NULL	NULL	NULL	NULL

```
175 • SELECT *
176 FROM Employees
177 LIMIT 10 OFFSET 3;
```

Result Grid

 Filter Rows:

Edit:  

	employee_id	name	role	salary	hire_date
	4	Priya Singh	HR	55000	2020-11-05
	5	Karan Patel	Developer	62000	2022-06-18
	6	Sneha Gupta	Tester	48000	2023-09-01
	7	Rohit Jain	Support	40000	2021-12-12
	8	Anjali Desai	Analyst	53000	2022-08-25
	NULL	NULL	NULL	NULL	NULL



8. Joins



INNER JOIN

Return matching rows from both tables

```
SELECT c.name, o.order_id  
FROM Customers c  
INNER JOIN Orders o  
ON c.customer_id = o.customer_id;
```



LEFT JOIN

Return all rows from left table, matching from right if available

```
SELECT c.name, o.order_id FROM Customers c  
LEFT JOIN Orders o  
ON c.customer_id = o.customer_id;
```



RIGHT JOIN

Return all rows from right table, matching from left if available)

```
SELECT c.name, o.order_id FROM  
Customers c  
RIGHT JOIN Orders o  
ON c.customer_id = o.customer_id;
```




8. Joins



FULL JOIN

A FULL JOIN returns all rows from both tables, with matching rows combined when possible. If there's no match, it shows NULL.



SELF JOIN

A SELF JOIN is when a table is joined with itself. It's useful for comparing rows in the same table

8. Joins



```
181  -- Inner join --
182
183 •  SELECT c.name, o.order_id, o.total_amount
184     FROM Customers c
185     INNER JOIN Orders o
186     ON c.customer_id = o.customer_id;
187
```

Result Grid	Filter Rows:	Export:
name	order_id	total_amount
Amit Verma	1	450
Neha Gupta	2	299
Rahul Joshi	3	650
John Doe	4	200
Amit Verma	5	780
Neha Gupta	6	120
Jane Smith	7	540
Rahul Sharma	8	310

```
190  -- Left Join
191 •  SELECT c.name, o.order_id, o.total_amount
192     FROM Customers c
193     LEFT JOIN Orders o
194     ON c.customer_id = o.customer_id;
195
```

Result Grid	Filter Rows:	Export:
name	order_id	total_amount
Amit Verma	1	450
Amit Verma	5	780
Neha Gupta	2	299
Neha Gupta	6	120
Rahul Joshi	3	650
John Doe	4	200
Jane Smith	7	540
Rahul Sharma	8	310
Priya Patel	NULL	NULL



```
198  -- right join
199 •  SELECT c.name, o.order_id, o.total_amount
200     FROM Customers c
201     RIGHT JOIN Orders o
202     ON c.customer_id = o.customer_id;
203
```

Result Grid	Filter Rows:	Export:
name	order_id	total_amount
Amit Verma	1	450
Neha Gupta	2	299
Rahul Joshi	3	650
John Doe	4	200
Amit Verma	5	780
Neha Gupta	6	120
Jane Smith	7	540
Rahul Sharma	8	310

8. Joins



```
205 -- full outer join
206 • SELECT c.name, o.order_id, o.total_amount
207 FROM Customers c
208 LEFT JOIN Orders o
209 ON c.customer_id = o.customer_id
210
211 UNION
212
213 SELECT c.name, o.order_id, o.total_amount
214 FROM Customers c
215 RIGHT JOIN Orders o
216 ON c.customer_id = o.customer_id;
217
```

Result Grid	Filter Rows:	Export:
name	order_id	total_amount
Amit Verma	1	450
Amit Verma	5	780
Neha Gupta	2	299
Neha Gupta	6	120
Rahul Joshi	3	650
John Doe	4	200
Jane Smith	7	540
Rahul Sharma	8	310
Priya Patel	NULL	NULL

```
229 -- CROSS JOIN
230 • SELECT c.name, e.name AS employee_name
231 FROM Customers c
232 CROSS JOIN Employees e;
```

Result Grid

```
218 -- self join
219
220 • SELECT e1.name AS employee1, e2.name AS employee2, e1.role
221 FROM Employees e1
222 JOIN Employees e2
223 ON e1.role = e2.role
224 AND e1.employee_id <> e2.employee_id;
225
```

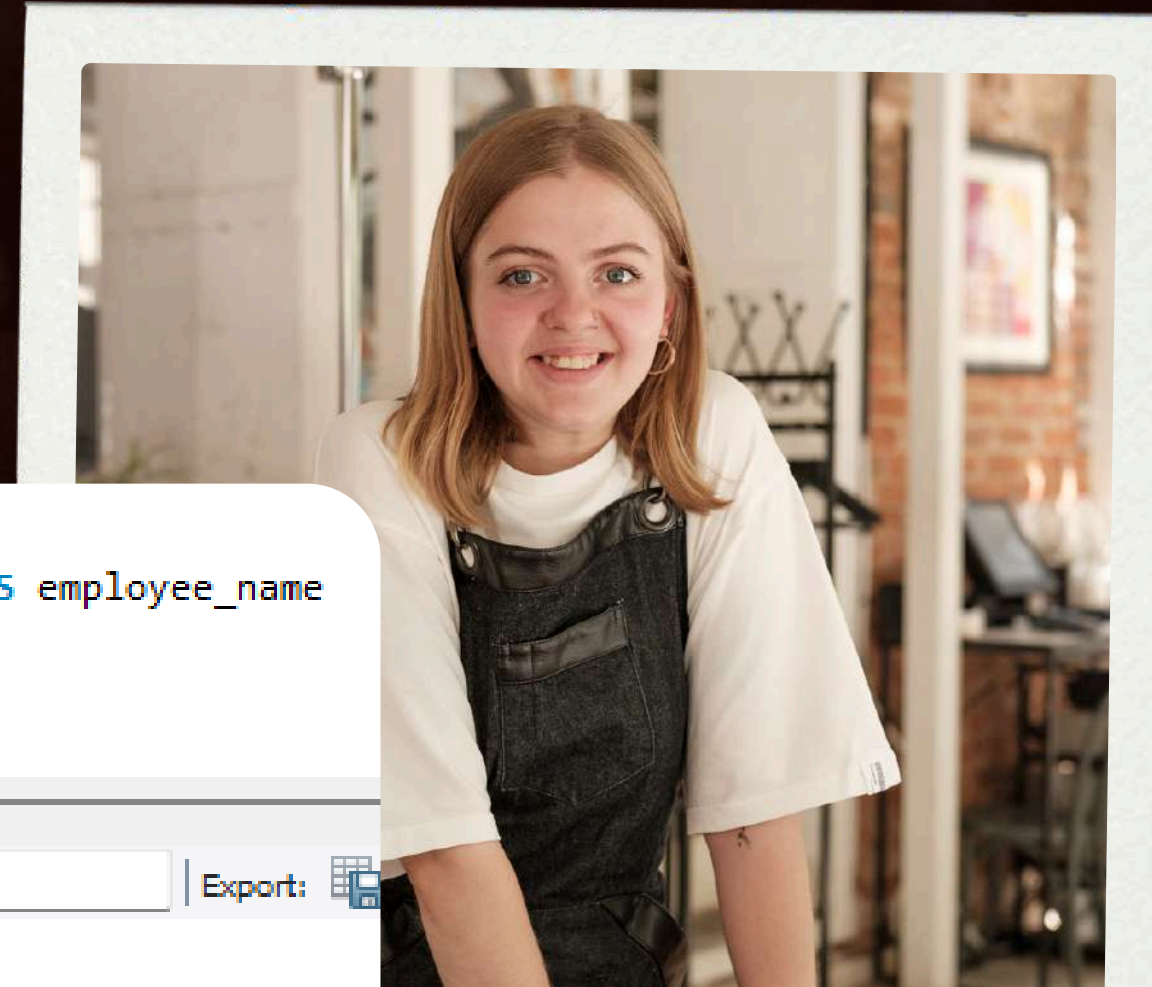
Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	employee1	employee2	role
	Karan Patel	Neha Verma	Developer
	Neha Verma	Karan Patel	Developer





9. Advance SQL

- **CTE (Common Table Expression)** :
Temporary named result set for complex queries.

```
236 • WITH CustomerOrders AS (  
237     SELECT customer_id, SUM(total_amount) AS total_spent  
238     FROM Orders  
239     GROUP BY customer_id  
240 )  
241 SELECT *  
242 FROM CustomerOrders;  
243
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
customer_id	total_spent		
1	1230		
2	419		
3	650		
4	200		

```
WITH CustomerOrders AS (  
    SELECT customer_id, SUM(total_amount) AS total_spent  
    FROM Orders  
    GROUP BY customer_id  
)  
SELECT *  
FROM CustomerOrders;
```




9. Advance SQL

- Index :

Improves query speed by allowing faster searches on large tables

```
CREATE INDEX idx_customers_email  
ON Customers(email);
```

-- Advance Sql

```
CREATE INDEX idx_customers_email  
ON Customers(email);
```

```
SELECT *  
FROM Customers  
WHERE email = 'amit.verma@example.com';
```




10. CONCLUSION

In this project, we explored essential SQL operations using a coffee shop database. We performed CRUD operations, filtered records with WHERE and LIKE, and used aggregation functions like SUM, COUNT, and AVG. We practiced sorting, pagination, and various joins (INNER, LEFT, RIGHT, FULL, SELF, CROSS), and applied advanced concepts like CTEs, window functions, and indexes. These techniques help a coffee shop analyze customer behavior, track orders, and make data-driven business decisions efficiently.



Thank You

We appreciate your time and interest in our Coffee Shop Project.

This project demonstrates how SQL can turn raw data into actionable insights, helping businesses understand customers, track sales, and optimize operations.

Presented by: Dhanvanti Chelani

Date: 02-04-2026

